ITC CYBER SECURITY TEAM

Threat Modeling Challenge

Official Submission Report

Member Name	Team Name
Dehili Mohammed	Team 11
Theme	**Discord Username**
Health Tech	md1mohammed
Submission Date	
March 9, 2026	

March 9, 2026 | Discord Submission | 4-Day Deadline

Think. Analyze. Secure.

01 System Description

MediLink is a full-stack Health Tech platform built with React 19 (frontend) and Laravel 12 / PHP 8.2 (backend), designed to streamline healthcare operations.

Users: Patients · Doctors · Lab Technicians · Administrators (4 role-based interfaces).
Data handled: PHI (patient records, lab results, prescriptions, appointments), authentication tokens, secure messages, audit logs.
Components: React SPA (frontend) | Laravel REST API (backend) | MySQL database via Eloquent ORM | Laravel Sanctum (JWT auth) | Queue system | Email notifications.
Security baseline: RBAC middleware, bcrypt password hashing, CSRF protection, input sanitization, audit logging, HIPAA-compliant messaging.

02 Asset List

Asset Name	Asset Type	Why It Matters
Patient Health Records	Data (PHI)	Breach violates HIPAA; exposes medical history, diagnoses and prescriptions of all patients
Auth Tokens (Sanctum)	Credential	Stolen token allows full impersonation of any user role without re-authentication
MySQL Database	Data Store	Central repository of all PHI; total compromise equals a full-scale data breach
Secure Messaging System	Service	Interception violates doctor-patient confidentiality and HIPAA communication requirements
Lab Results & Documents	Data (PHI)	Tampered or falsified results can cause misdiagnosis, incorrect treatment, or patient harm
Laravel API Backend	Service	API is the single gateway to all data and actions; compromise grants access to the entire system

03 Threat Analysis

ID	Threat Name	STRIDE Type	Target Component	Description
T1	Token / Session Hijacking	Spoofing	Sanctum Auth API	Attacker injects XSS payload to steal a Sanctum bearer token and impersonate any authenticated user
T2	SQL Injection Attack	Tampering	MySQL Database	Malicious SQL crafted in API inputs bypasses Eloquent ORM on raw queries, dumping full PHI tables
T3	Broken Access Control (RBAC)	Elevation of Privilege	RBAC Middleware	Patient manipulates the role parameter in API requests to access restricted /admin or /doctor routes
T4	Stored XSS in Messaging	Tampering	Messaging Module	Injected script in a chat message persists in the DB and executes in every recipient's browser session
T5	Insecure File Upload (Lab)	Tampering	Lab Upload Module	Malicious PHP web shell uploaded via lab results endpoint enables remote code execution on the server
T6	API Rate Limit Abuse (DoS)	Denial of Service	Laravel API Backend	Automated flooding of unauthenticated endpoints crashes the server and denies service to all users
T7	Audit Log Tampering	Repudiation	Admin Audit Logs	Insider threat deletes or alters audit log entries to cover unauthorized access to sensitive patient data
T8	Verbose Error Info Leakage	Information Disclosure	API Error Responses	Unhandled exceptions return stack traces, DB schema details and internal paths that aid further attacks

04 Risk Assessment Table

ID	Threat Name	Likelihood	Impact	Risk Level	Justification
T1	Token Hijacking	High	Critical	Critical	Full account takeover; attacker gains all data and actions of the hijacked role
T2	SQL Injection	Medium	Critical	High	Eloquent ORM reduces exposure but raw queries in report endpoints remain vulnerable
T3	Broken Access Control	High	Critical	Critical	Role confusion allows any patient to invoke admin-only operations and view all PHI
T4	XSS in Messaging	Medium	High	High	Large chat surface and React's innerHTML usage increases stored-XSS attack probability
T5	Insecure File Upload	Medium	Critical	High	No MIME-type validation detected in lab upload module; RCE via web shell is viable
T6	API Rate Limit Abuse	High	High	High	No throttle middleware visible on public endpoints; easy to flood with automated scripts
T7	Audit Log Tampering	Low	High	Medium	Requires privileged insider access but absence of write-once logs makes tampering trivial
T8	Verbose Error Leakage	High	Medium	Medium	Debug mode may be active in staging/production; stack traces expose internal architecture

05 Mitigation Plan

ID	Threat Name	Security Control	Control Type	Justification
T1	Token Hijacking	Enforce Content Security Policy headers; set Sanctum tokens to HttpOnly cookies	Preventive	CSP blocks inline script execution; HttpOnly prevents JavaScript from reading the token
T2	SQL Injection	Mandate Eloquent ORM for all queries; ban raw DB::statement calls in code reviews	Preventive	Parameterized queries eliminate the SQL injection surface at the framework level
T3	Broken Access Control	Add server-side RBAC middleware on every API route; validate role server-side only	Preventive	Server-enforced checks block cross-role access regardless of any client-side manipulation
T4	XSS in Messaging	HTML-encode all user-generated content on render; enable React's default XSS escaping	Preventive	Output encoding neutralises injected scripts before the browser can execute them
T5	Insecure File Upload	Whitelist MIME types (PDF/images only); rename files server-side; store outside webroot	Preventive	Type whitelist blocks executables; renaming and isolated storage prevent web shell access
ID	Threat Name	Security Control	Control Type	Justification
T6	API Rate Limit Abuse	Deploy Laravel throttle middleware (e.g. 60 req/min per IP) on all public endpoints	Preventive	Rate caps deter automated flood attacks; 429 responses are returned to slow/block bots

06 Conclusion

MediLink'sthreatmodelidentified**8threats**acrossallsixSTRIDEcategoriestargetingitsReact19+Laravel12 stack.

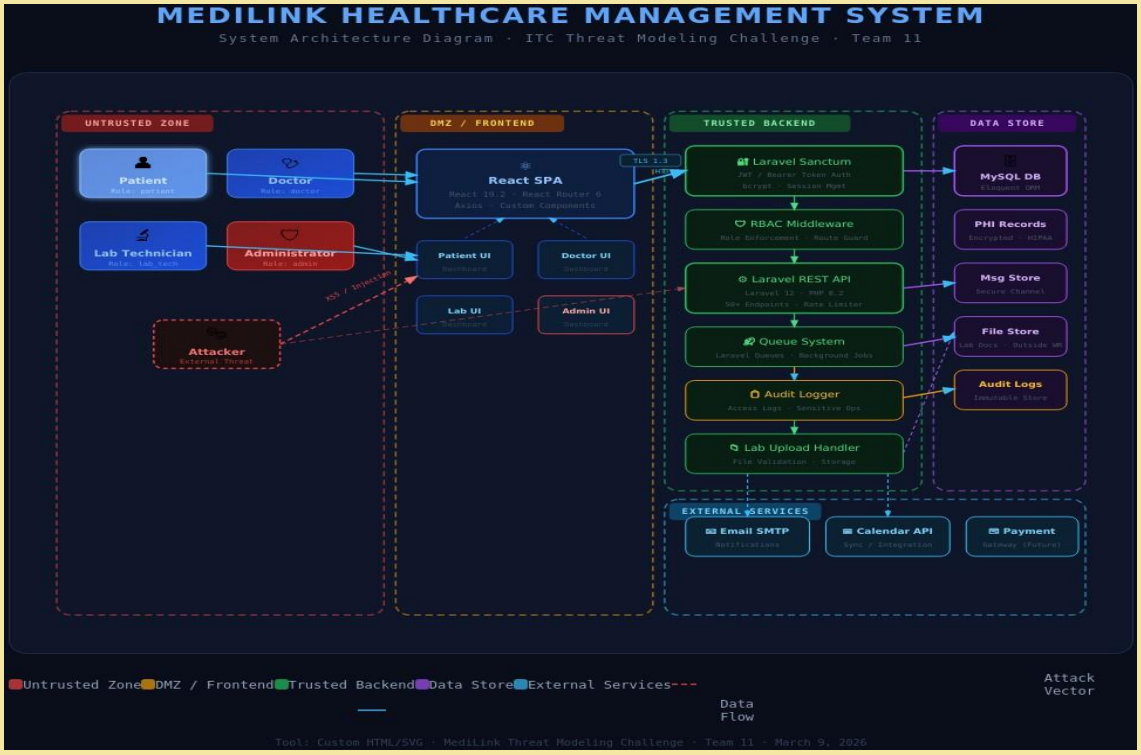
CriticalThreats(immediateactionrequired):T1–Token/SessionHijacking,T3–BrokenAccessControl. **HighThreats:**T2–SQLInjection,T4–StoredXSS,T5–InsecureFileUpload,T6–APIRateLimitAbuse. **Medium Threats:** T7 – Audit Log Tampering, T8 – Verbose Error Leakage.

OverallSecurityPosture:MODERATE. Laravel'sEloquentORMandSanctumprovideasolidauthentication baseline. However, the absence of Content Security Policy headers, missing MIME-type validation on lab uploads, and no observable rate-limiting middleware create directly exploitable attack paths.

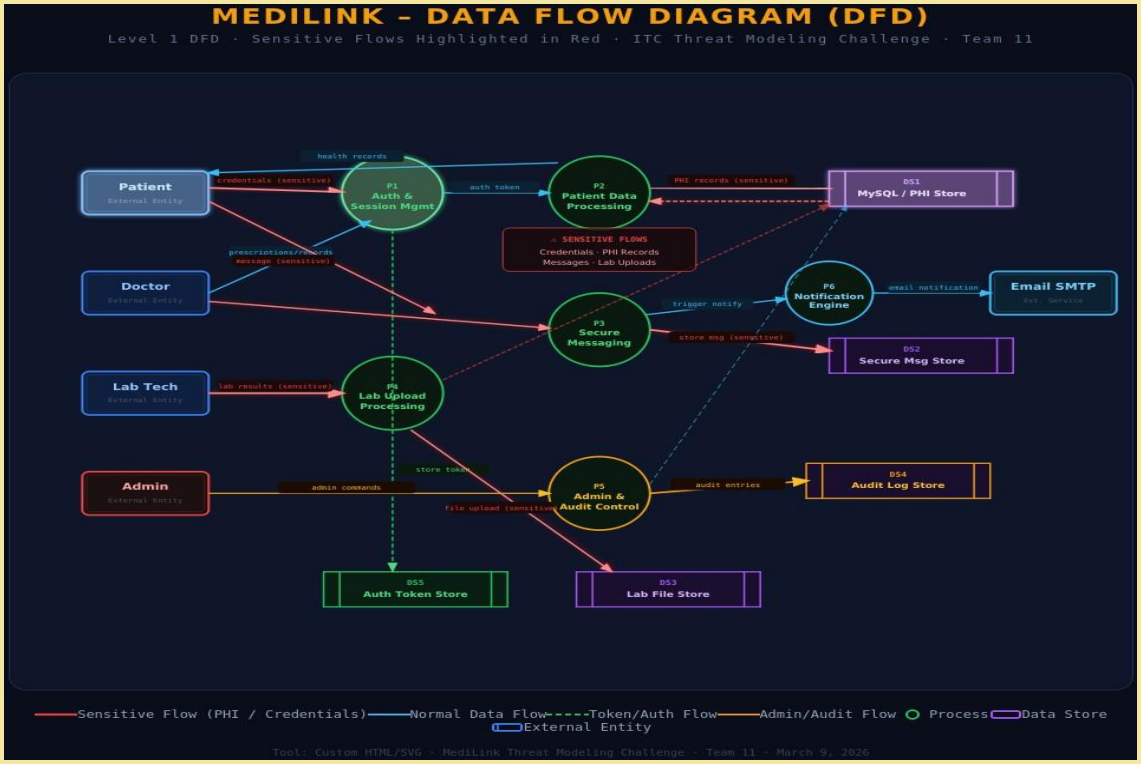
Top Priority Actions:
1. Implement CSP headers and HttpOnly cookie storage for Sanctum tokens to prevent T1 token theft via XSS.
2. Enforce server-side RBAC middleware on every API endpoint to close T3 privilege escalation paths.
3. AddMIME-typewhitelistingandserver-sidefilerenamingtothelabuploadmoduletopreventT5remotecode execution.
4. Deploy Laravel throttle middleware on all public-facing endpoints to mitigate T6 denial-of-service risk.

AddressingthesefourcontrolswouldraiseMediLink'ssecuritypostureto**STRONG**andbringtheplatforminto full HIPAA compliance.

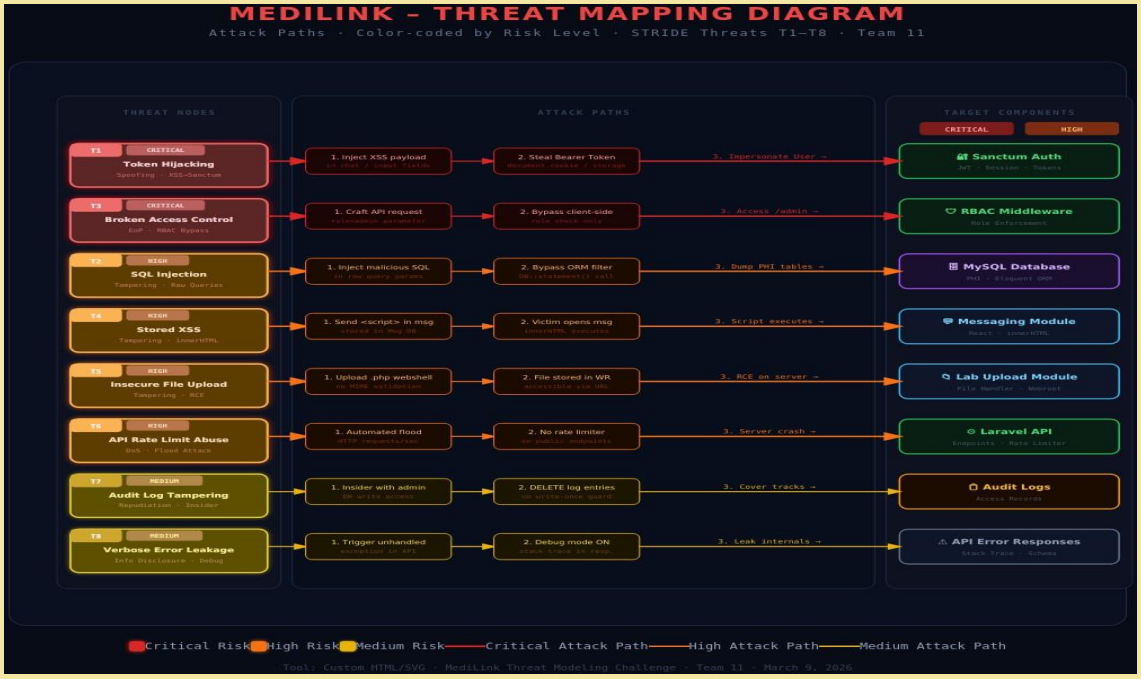
07 System Architecture Diagram



08 Data Flow Diagram (DFD)



09 Threat Mapping Diagram



✓ Submission Checklist

#	Deliverable	Type	Included?
1	System Description		■
2	Asset List	Written	■
3	Threat Analysis		■
4	Risk Assessment Table	Written	■
5	Mitigation Plan		■
6	Conclusion	Written	■
7	System Architecture Diagram	DIAGRAM	■
8	Data Flow Diagram (DFD)	DIAGRAM	■
9	Threat Mapping Diagram	DIAGRAM	■

Think. Analyze. Secure.

ITC Cyber Security Team | Threat Modeling Challenge | March 9, 2026